UMI AI CHATBOT

System Architecture, Installation Guide, and Technical Reference

1. System Overview & Packages Used

The UMI AI Chatbot is an enterprise-grade assistant built using Django on the backend and a modern, clean layout on the frontend. It supports connection pooling, local memory caching, and progressive streaming for Ollama and Gemini.

Package	Version	Description / Use Case
Django	6.0.6	Core MVC/MTV framework, ORM engine, forms handling, and views architecture.
google-genai	2.10.0	Official SDK for integration with Google Gemini 2.5 models.
rich	15.0.0	Terminal logging, console formatting, traceback catching, and status panel display.
requests	2.34.2	HTTP client library used for communication with local Ollama API endpoints.
python-dotenv	1.2.2	Loads settings keys and sensitive parameters securely from a local .env file.
pypdf	6.14.2	Extracts text content programmatically from uploaded PDF files.
python-docx	1.2.0	Parses text data from Microsoft Word documents (.docx) uploads.
openpyxl	3.1.5	Reads and extracts values from Microsoft Excel worksheets (.xlsx).
markdown	3.10.2	Converts AI raw responses from markdown format to HTML elements.

2. Installation Guide

Follow these steps to deploy and run the UMI Chatbot locally:

1. **Step 1: Install Ollama (For Local Models)**
Download and install Ollama from the official website. Launch the server in your terminal:
ollama run qwen2.5:0.5b

2. **Step 2: Clone and Navigate**
Clone the project directory to your workspace: cd c:/AI_ChatBot

3. **Step 3: Setup Virtual Environment**
Create and activate a python virtual environment:
python -m venv venv
.\venv\Scripts\activate

4. **Step 4: Install Dependencies**
Install python modules from the requirements configuration:
pip install -r requirements.txt

5. **Step 5: Configure Environment Keys**
Create a .env file in the root folder containing your Gemini API key:
GEMINI_API_KEY=your_actual_key_here

6. **Step 6: Run Migrations and Launch**
Synchronize your database schema and start the local development server:
python manage.py migrate
python manage.py runserver

3. Core Function Reference

Below is the complete reference documentation for UMI's primary functions in the view, AI service, and helper modules.

chat/views.py

get_sidebar_context(user)

Generates a categorized dictionary of conversation history for the sidebar (Today, Yesterday, Last 7 Days, Older). Utilizes LocMemCache caching to avoid redundant DB queries, and queries pinned chats in a single query to eliminate N+1 overhead.

send_message(request, chat_id)

The main controller views that receives user prompts and attachments. Prepares history context using a sliding window of the last 10 messages, fetches cached file text to skip repeated disk parsing, streams response text via StreamingHttpResponse, and spawns the async summarizer thread at completion.

invalidate_sidebar_cache(user_id)

Clears the cached sidebar context dictionary from memory whenever conversation status changes (renames, deletions, pins, preferences).

run_async_summarizer(conversation_id, provider, model)

Spawns a daemon background thread (`threading.Thread`) to summarize messages older than the last 10 turns. Caches the resulting summary text to minimize subsequent request token sizes.

upload_file(request)

Handles multipart form AJAX file uploads. Validates the file extension, stores the file in the attachments media folder, registers an Attachment model record, and returns the metadata JSON.

chat/ai_service.py

get_gemini_client(api_key)

Fetches or creates a cached `genai.Client` client instance. Reusing the client maintains active TCP connection pools for fast streaming response performance.

get_ollama_models()

Queries the local Ollama `/api/tags` endpoint to fetch local brains. Caches results in Django's memory cache for 1 hour to prevent pageload blocking.

get_system_prompt_for_style(style)

Compiles detailed instructions customized for the active mode (Short, Balanced, Detailed, Research). Guides formatting (headings, lists, tables) and step-by-step code analysis.

stream_ai_response(provider, model, prompt, history, system_prompt, multimodal_parts, response_style)

Primary streaming controller. Resolves model tokens budgets dynamically. Routes calls to Gemini or local Ollama using connection reuse.

chat/services.py

render_markdown_to_html(text)

Converts markdown strings into clean, safe HTML elements. Employs codehilite and fenced code blocks for syntax highlighting.

extract_text_from_file(file_path, file_name)

Detects file extension and extracts text programmatically. Supports PDF extraction via pypdf, Word extraction via docx, Excel cells via openpyxl, and CSV columns.